UNIOESTE – Universidade Estadual do Oeste do Paraná

Curso de Ciência da Computação
Disciplina: Projeto e Análise de Algoritmos (PAA)
Docentes: Prof. Dr. Rômulo e Prof. Dr. Newton Spolaôr
Discente: Vinícius Fontana Munchen
Trabalho Prático: Algoritmos em Grafos - *GraphLab* PAA

1. Descrição Geral e Funcionamento do Software

O *GraphLab PAA* é uma aplicação gráfica interativa desenvolvida em Java (Swing + JGraphX) para carregar, processar, analisar e visualizar grafos orientados e não-orientados ponderados.

1.1. Funcionalidades e Algoritmos Atendidos

1. **Leitura e Validação de Arquivo Texto:** *Parser* automático do formato oficial do trabalho (`orientado=sim/nao` , `V=<n>` , `(u,v):peso`).
2. **Representação com Listas de Adjacência Próprias:** Implementação manual de lista encadeada mantendo adjacências em ordem estritamente crescente.
3. **Desenho e Visualização Gráfica Interativa (Item a):** Renderização visual com nós rotulados, arestas ponderadas com direcionamento (setas), minimização de cruzamentos via *recozimento simulado* (*Simulated Annealing*) e curvas para arestas paralelas.
4. **Busca em Profundidade - DFS (Item b):** Travessia sistemática a partir de um vértice arbitrário, garantindo visitação de componentes desconexas. Formato de saída: `u - v - w`.
5. **Busca em Largura - BFS (Item c):** Exploração em camadas via fila circular própria, cobrindo todo o grafo mesmo se desconexo. Formato: `u - v - w`.
6. **Ordenação Topológica (Item d):** Ordenação de dependências em Grafos Orientados Acíclicos (DAGs) com detecção robusta de ciclos e verificação de orientação.
7. **Componentes Fortemente Conexas - Kosaraju (Item e):** Identificação de CFCs em grafos orientados usando grafo transposto, exibição em ordem decrescente de tamanho e coloração automática de cada componente com cores distintas na interface gráfica.
8. **Algoritmo de Dijkstra (Item f):** Cálculo dos menores caminhos a partir de uma origem em grafos direcionados com pesos não-negativos, com validação de pré-requisitos.
9. **Algoritmo de Kruskal - Árvore Geradora Mínima (Item g):** Determinação da AGM em grafos não-orientados com estrutura Union-Find própria (compressão de caminho e união por rank), cálculo do peso total e destaque visual das arestas selecionadas em vermelho e maior espessura.

1.2. Guia de Uso

1. **Inicialização:** Execute a aplicação (conforme instruções de build na Seção 3). A tela inicial apresentará o menu com botões desabilitados até a seleção de um grafo.
2. **Carregar Grafo:**
   - Clique em `Carregar Grafo (.txt)`.
   - Escolha o arquivo texto desejado (ex: `fig1.txt`, `fig2.txt`, `fig3.txt`).
   - A barra superior atualizará com o resumo do grafo (Arquivo, Vértices, Arestas, Orientação) e o grafo será desenhado imediatamente na tela central.
3. **Executar Algoritmos:**
   - Clique no botão do algoritmo desejado no menu lateral esquerdo.
   - **DFS, BFS e Dijkstra:** Uma caixa de diálogo solicitará o vértice de origem (entre `0` e `V-1`).
   - **Validação de Incompatibilidade:** Caso o grafo não atenda aos requisitos do algoritmo (ex: Kruskal em grafo orientado, Dijkstra com pesos negativos, Ordenação Topológica com ciclos), o sistema emitirá um aviso claro explicando o impedimento, conforme exigido pela especificação.
   - **Resultados:** A saída textual padronizada é impressa simultaneamente no terminal (`System.out`) e no console inferior da interface gráfica, com título identificador da operação.
   - **Destaques Visuais:** Algoritmos como Kosaraju e Kruskal atualizam dinamicamente as cores dos nós ou o destaque das arestas no painel gráfico. O botão `Desenhar grafo` restaura as cores padrão a qualquer momento.

2. Processo de Construção (Build) e Execução

2.1. Requisitos e Ferramentas

- **Linguagem:** Java SE (JDK 21 LTS recomendado; compatível com JDK 17+).
- **Biblioteca Gráfica Externa:** `lib/jgraphx.jar` (JGraphX v3.9.8.1 - 100% Java Swing puro, multiplataforma).
- **Ambiente de Desenvolvimento (IDE):** Visual Studio Code (1.144+).
- **Sistema Operacional:** Compatível com Windows 10/11, Linux (Ubuntu/Debian/Fedora) e macOS (Apple Silicon / Intel).

2.2. Estrutura Organizada do Projeto

```
t2-paa/
├── exemplos/
│   ├── fig1.txt                # Grafo orientado com pesos negativos (Figura 1)
│   ├── fig2.txt                # Grafo não-orientado ponderado (Figura 2)
│   └── fig3.txt                # Grafo não-orientado desconexo (Figura 3)
├── lib/
│   └── jgraphx.jar             # Biblioteca visual JGraphX (Java puro
multiplataforma)
```

```
├── out/                        # Bytecodes compilados (.class)
├── src/
│   ├── algorithms/             # DFS, BFS, Topológica, Kosaraju, Dijkstra,
Kruskal, Fachada
│   ├── io/                     # Leitor e parser sintático de arquivo texto
│   ├── model/                  # Entidades de domínio (Grafo e Aresta)
│   ├── structures/             # Estruturas próprias (ListaAdjacencia, FilaInt,
PilhaInt, UnionFind)
│   ├── ui/                     # Interface Swing (JanelaPrincipal) e Visualizador
(PainelVisualizacaoGrafo)
│   └── Main.java               # Ponto de entrada da aplicação
├── .gitignore                  # Regras de exclusão de artefatos temporários e IDE
├── README.doc                  # Relatório técnico formal para avaliação
```

2.3. Instruções de Compilação e Execução

- **No Windows:**

```
javac -cp "lib/jgraphx.jar" -d out src/structures/*.java src/model/*.java
src/io/*.java src/algorithms/*.java src/ui/*.java src/Main.java
java -cp "out;lib/jgraphx.jar" Main
```

- **No Linux e macOS:**

```
javac -cp "lib/jgraphx.jar" -d out $(find src -name "*.java")
java -cp "out:lib/jgraphx.jar" Main
```

3. Citação dos Trechos do Software (Estruturas e Algoritmos)

Conforme exigido pelo Item 3 da especificação, apresentam-se abaixo os trechos fundamentais implementados do zero para cada estrutura de dados e algoritmo.

3.1. Estruturas de Dados Próprias

3.1.1. Lista de Adjacência com Inserção Ordenada (`src/structures/ListaAdjacencia.java`)

Garante que os vizinhos fiquem sempre ordenados crescentemente pelo índice:

```
public void inserirOrdenado(int destino, int peso) {
    NoAdjacencia novo = new NoAdjacencia(destino, peso);
    if (cabeca == null || cabeca.getDestino() > destino) {
        novo.setProximo(cabeca);
        cabeca = novo;
    } else {
        NoAdjacencia atual = cabeca;
        while (atual.getProximo() != null && atual.getProximo().getDestino() <
destino) {
            atual = atual.getProximo();
        }
        novo.setProximo(atual.getProximo());
```

```
            atual.setProximo(novo);
        }
    }
```

3.1.2. Fila Circular Manual (`src/structures/FilaInt.java`)

Implementada para a Busca em Largura (BFS) sem uso de coleções prontas:

```
public void enfileirar(int valor) {
    if (estaCheia()) throw new IllegalStateException("Capacidade máxima da fila
excedida.");
    tras = (tras + 1) % dados.length;
    dados[tras] = valor;
    tamanho++;
}

public int desenfileirar() {
    if (estaVazia()) throw new IllegalStateException("Fila vazia.");
    int valor = dados[frente];
    frente = (frente + 1) % dados.length;
    tamanho--;
    return valor;
}
```

3.1.3. Pilha Estática (`src/structures/PilhaInt.java`)

Utilizada para DFS recursiva, Ordenação Topológica e Kosaraju:

```
public void empilhar(int valor) {
    if (estaCheia()) throw new IllegalStateException("Pilha cheia.");
    dados[++topo] = valor;
}

public int desempilhar() {
    if (estaVazia()) throw new IllegalStateException("Pilha vazia.");
    return dados[topo--];
}
```

3.1.4. Union-Find com Compressão de Caminho e Rank (`src/structures/UnionFind.java`)

Garante operações de conjunto quase lineares $O(\alpha(V))$ para o algoritmo de Kruskal:

```
public int encontrar(int i) {
    if (pai[i] != i) {
        pai[i] = encontrar(pai[i]); // Compressão de caminho
    }
    return pai[i];
}

public boolean unir(int x, int y) {
```

```
    int raizX = encontrar(x);
    int raizY = encontrar(y);
    if (raizX == raizY) return false;

    if (rank[raizX] < rank[raizY]) {
        pai[raizX] = raizY;
    } else if (rank[raizX] > rank[raizY]) {
        pai[raizY] = raizX;
    } else {
        pai[raizY] = raizX;
        rank[raizX]++;
    }
    return true;
}
```

3.2. Algoritmos em Grafos

3.2.1. Busca em Profundidade (`src/algorithms/AlgoritmoDFS.java`)

```
public ResultadoAlgoritmo executar(Grafo grafo, int origem) {
    // ... validação de limites ...
    boolean[] visitado = new boolean[n];
    int[] ordem = new int[n];
    int[] contador = { 0 };

    dfsRecursivo(grafo, origem, visitado, ordem, contador);
    for (int i = 0; i < n; i++) {
        if (!visitado[i]) dfsRecursivo(grafo, i, visitado, ordem, contador);
    }
    return ResultadoAlgoritmo.sucesso(ResultadoAlgoritmo.formatarSequencia(ordem,
contador[0]));
}

private void dfsRecursivo(Grafo grafo, int u, boolean[] visitado, int[] ordem, int[]
contador) {
    visitado[u] = true;
    ordem[contador[0]++] = u;
    for (NoAdjacencia at = grafo.getAdjacencia(u).getCabeca(); at != null; at =
at.getProximo()) {
        if (!visitado[at.getDestino()]) {
            dfsRecursivo(grafo, at.getDestino(), visitado, ordem, contador);
        }
    }
}
```

3.2.2. Busca em Largura (`src/algorithms/AlgoritmoBFS.java`)

```
private int explorarComponente(Grafo grafo, int inicio, boolean[] visitado, int[]
ordem, int contador, FilaInt fila) {
    visitado[inicio] = true;
    fila.enfileirar(inicio);
```

```
    while (!fila.estaVazia()) {
        int u = fila.desenfileirar();
        ordem[contador++] = u;
        for (NoAdjacencia at = grafo.getAdjacencia(u).getCabeca(); at != null; at =
at.getProximo()) {
            int v = at.getDestino();
            if (!visitado[v]) {
                visitado[v] = true;
                fila.enfileirar(v);
            }
        }
    }
    return contador;
}
```

3.2.3. Ordenação Topológica com Detecção de Ciclos (`src/algorithms/AlgoritmoOrdenacaoTopologica.java`)

```
private boolean possuiCiclo(Grafo grafo, int u, int[] estado, PilhaInt pilha) {
    estado[u] = 1; // 1 = em processamento na pilha recursiva
    for (NoAdjacencia at = grafo.getAdjacencia(u).getCabeca(); at != null; at =
at.getProximo()) {
        int v = at.getDestino();
        if (estado[v] == 1 || (estado[v] == 0 && possuiCiclo(grafo, v, estado,
pilha))) {
            return true;
        }
    }
    estado[u] = 2; // 2 = finalizado
    pilha.empilhar(u);
    return false;
}
```

3.2.4. Componentes Fortemente Conexas - Kosaraju (`src/algorithms/AlgoritmoKosaraju.java`)

Executa DFS no grafo original para obter os tempos de término, inverte as arestas construindo o transposto G^T, e executa nova DFS coletando as componentes na ordem inversa:

```
private Grafo construirTransposto(Grafo g) {
    Grafo gT = new Grafo(g.getNumVertices(), true);
    for (int u = 0; u < g.getNumVertices(); u++) {
        for (NoAdjacencia at = g.getAdjacencia(u).getCabeca(); at != null; at =
at.getProximo()) {
            gT.adicionarAresta(at.getDestino(), u, at.getPeso());
        }
    }
    return gT;
}
```

3.2.5. Algoritmo de Dijkstra (`src/algorithms/AlgoritmoDijkstra.java`)

Relaxamento guloso de arestas partindo da menor distância conhecida:

```
for (int count = 0; count < n; count++) {
    int u = -1, menor = INFINITO;
    for (int v = 0; v < n; v++) {
        if (!fechado[v] && dist[v] < menor) {
            menor = dist[v];
            u = v;
        }
    }
    if (u == -1) break;
    fechado[u] = true;

    for (NoAdjacencia at = grafo.getAdjacencia(u).getCabeca(); at != null; at =
at.getProximo()) {
        int v = at.getDestino();
        if (!fechado[v] && dist[u] != INFINITO && dist[u] + at.getPeso() < dist[v])
{
            dist[v] = dist[u] + at.getPeso();
        }
    }
}
```

3.2.6. Algoritmo de Kruskal (`src/algorithms/AlgoritmoKruskal.java`)

Ordena arestas por peso e une conjuntos disjuntos caso não formem ciclo:

```
for (int i = 0; i < m && qtdAgm < n - 1; i++) {
    Aresta a = ordenadas[i];
    if (uf.unir(a.getU(), a.getV())) {
        a.setDestacada(true);
        agm[qtdAgm++] = a;
        pesoTotal += a.getPeso();
    }
}
```

4. Referências Bibliográficas

1. **JGRAPHX LIBRARY.** *JGraphX Java Graph Visualization and Layout Library.* Disponível em: https://github.com/jgraph/jgraphx.
2. **DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL GENERATIVA**
   - *Ferramenta utilizada:* Antigravity (Google DeepMind).
   - *Finalidade:* Apoio na refatoração e otimização da concisão do código-fonte Java, estruturação da folha de estilos do JGraphX.
   - *Supervisão e Responsabilidade:* Todos os trechos gerados ou sugeridos foram revisados, testados contra os grafos do trabalho (Figuras 1, 2 e 3), depurados e validados

integralmente.